



CSE 481: CONTROL ENGINEERING

MAJOR TASK

Instructor: Dr Micheal Ibrahim

TA: Eng Mina Maher

Made By:

Karim Samer Mostafa	23p0439
Omar Gamil Anwar	23p0438
Tarek Mohamed Atef	2300245
Paula Hanna Nagib	23p0440
Youssef Maged Morees	2301081



Contents

1 Introduction	2
2 Project Objective	2
3 System Overview and Working Principle	2
4 Hardware Components	3
5 Mechanical Assembly and Pointer Design	5
6 Wiring Connections	6
6.1 Arduino to L298N Driver.....	6
6.2 Encoder to Arduino	6
6.3 Motor and Power Wiring.....	7
7 Closed-Loop Control System Description	7
7.1 Block Diagram	7
7.2 Control System Elements	8
8 Encoder Feedback and PPR Calibration	8
8.1 Zeroing Procedure	8
8.2 PPR Estimation Procedure	9
10 Motor Mathematical Model	10
11 Parameter Estimation	10
Nominal Parameter Values	10
Estimation Using Simulink.....	11
Model Validation and H-Bridge Dynamics.....	13
Final Estimated Parameters	14
12 MATLAB/Simulink Simulation	15
12.1 Motor Model	15
12.2 PID Motor Model.....	15
12.3 Full System for Tuning	15
12.4 Simulation vs. Hardware	16
13 PID Controller Design	17
13.1 Integral Anti-Windup.....	17
13.2 Minimum PWM Compensation.....	17
13.3 Target Tolerance and Settling Monitor.....	17
14 Final Arduino Implementation	17
14.1 Main Features	18

14.2 Serial Command Interface.....	18
14.3 Final Code Constants	18
15 Experimental PID Tuning Trials	18
15.1 Trial Summary Table	18
15.2 Trial 1.....	19
15.3 Trial 2.....	19
15.4 Trial 3.....	20
15.5 Trial 4.....	20
16 Final Response and Discussion	22
16.1 Final Response Metrics	22
16.2 Discussion of Final Performance	22
17 Problems Faced and Solutions	23
17.1 Encoder Wiring and Direction Issues	23
17.2 PPR Confusion.....	23
17.3 Model and Hardware Mismatch.....	23
17.4 Falling Response Mismatch During Parameter Estimation	23
17.5 Motor Stiction and Dead Zone	23
17.6 Integral Windup	23
17.7 Incremental Encoder Zero Position	23
18 Conclusion.....	24
Appendix A Final Arduino PID Code (PID.ino).....	25
Appendix B PPR Estimation Code (PPREstimation.ino)	27
Appendix C Parameter Estimation Code (ParameterEstimation.ino)	28

1 Introduction

Position control is one of the most common control applications in mechatronic and robotic systems. In a position control problem, the controller is required to move an actuator to a desired angular or linear position and maintain that position despite disturbances, friction, sensor noise, or modelling errors.

In this project, the controlled plant is a geared DC motor. A DC motor naturally converts electrical energy into rotational mechanical motion. However, without feedback, the motor cannot accurately stop at a desired angular position because its motion depends on supply voltage, load, friction, inertia, and driver losses. Therefore, a closed-loop feedback control system is required.

The implemented system uses encoder feedback to continuously estimate the motor shaft position. The measured position is compared with the desired target angle, and the resulting error is processed by a PID controller. The controller output is applied to the motor through an L298N motor driver using PWM and direction signals.

2 Project Objective

The main objective of this project is to design and implement a closed-loop angular position control system for a geared DC motor. The system should allow the motor output shaft to rotate to a required angular position and stop accurately without continuous oscillation.

- Construct a complete hardware setup for DC motor position control.
- Use an Arduino Uno to implement the controller.
- Use encoder feedback to measure the motor output shaft position.
- Convert encoder counts into angular position in degrees.
- Design and tune a PID controller for accurate position control.
- Use an L298N H-Bridge driver to control motor speed and direction.
- Validate the motor position physically using a pointer and a protractor.
- Perform open-loop data collection and parameter estimation for motor modelling.
- Simulate the system in MATLAB/Simulink.
- Compare PID tuning trials and select suitable PID parameters.

3 System Overview and Working Principle

The system is a closed-loop feedback control system. The user first aligns the physical pointer to zero degrees on the protractor and sends a zeroing command through the Serial Monitor. Since the encoder is incremental, this software zeroing step is required every time the system starts.

After zeroing, the user sends a relative movement command in the form:

```
angle direction
```

Examples:

```
90 0 → move the motor by +90° from current position
90 1 → move the motor by -90° from current position
```

The Arduino reads the current angle from the encoder count using the following formula, where the experimentally calibrated value $TOTAL_PPR = 995$ is used:

$$\theta = (\text{encoder_count} / TOTAL_PPR) \times 360^\circ$$

The error is computed at every sample step and passed through the discrete PID control law:

$$e(t) = \theta_{ref}(t) - \theta(t)$$

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t) dt + K_d \cdot (de(t)/dt)$$

The control output is saturated between -12 V and $+12\text{ V}$, converted to PWM in the range 0–255, and sent to the L298N driver. The sign of the output determines rotation direction; the magnitude determines the PWM duty cycle.

4 Hardware Components

The final hardware setup consists of the following components:

Component	Model / Type	Function in the System
Microcontroller	Arduino Uno Rev3	Runs the PID algorithm, reads encoder pulses, generates PWM and direction signals.
Motor Driver	L298N H-Bridge Module	Drives the DC motor in both directions using Arduino control signals.
DC Motor	GA25-370 Geared DC Motor with Encoder, 12 V, 130 RPM	The plant/actuator — produces the angular motion.
Encoder	Integrated incremental quadrature encoder	Provides digital pulse feedback used to estimate angular position.
Power Supply	12 V DC adapter	Provides external power to the motor through the L298N driver.
Mechanical Pointer	3D-printed PLA pointer	Mounted on the motor shaft to visually indicate angular position.
Visual Scale	Protractor	Used to verify the real output angle physically.
Wiring	Jumper wires	Connects Arduino, encoder, motor driver, and power supply.
USB Cable	Arduino USB cable	Used for programming and serial communication.

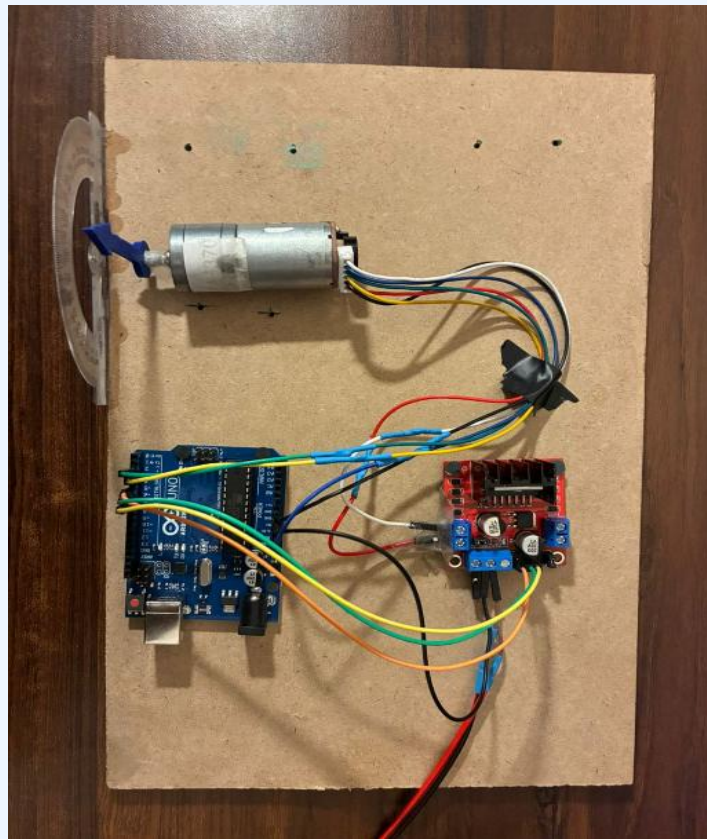


Figure 1: Overall hardware setup of the closed-loop DC motor position control system.

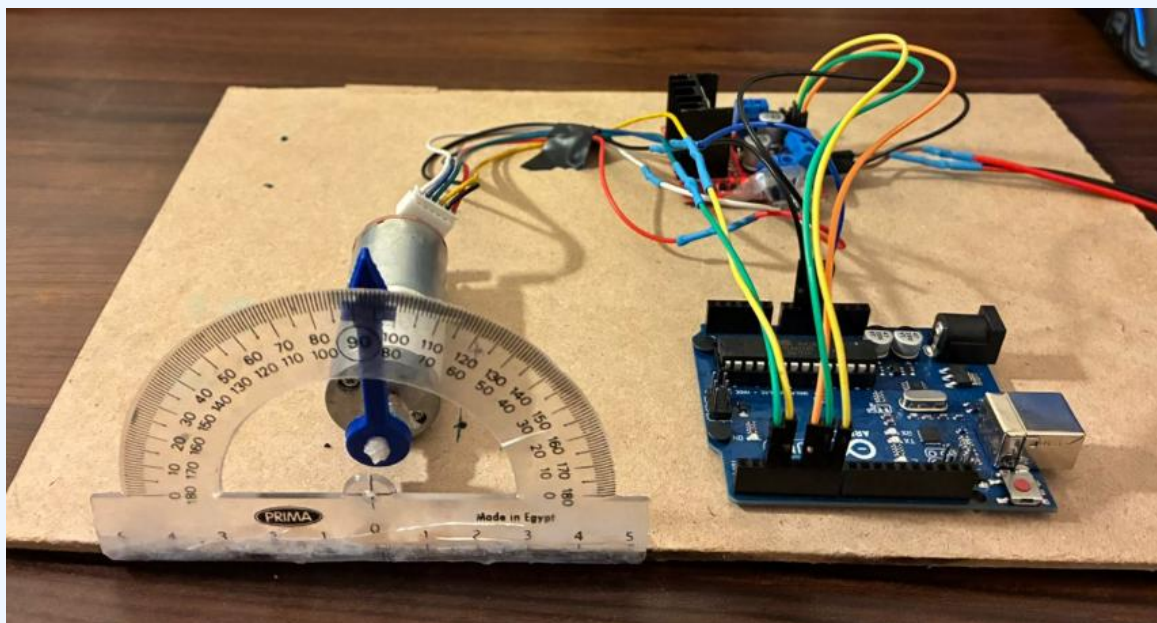


Figure 2: Main hardware components used in the project.

5 Mechanical Assembly and Pointer Design

The motor shaft was connected to a 3D-printed pointer. The pointer is used with a protractor to visually verify the actual angular displacement of the motor output shaft.

The pointer was designed to fit the measured motor shaft dimensions while allowing for realistic 3D-printing tolerances. A flat cut was included to match the motor shaft shape. Because the pointer was intentionally designed with a tolerance gap, adhesive was used to secure it firmly and prevent slipping during rotation.

Parameter	Value
Measured shaft diameter	4 mm
Measured shaft flat cut length	2.6 mm
Pointer hole diameter	6 mm
Pointer flat cut size	2.8 mm
Total pointer length	5 cm
Printing material	PLA
Infill	20%
Layer height	0.2 mm
Supports	Not required



Figure 3: 3D-printed pointer design used for visual angle indication

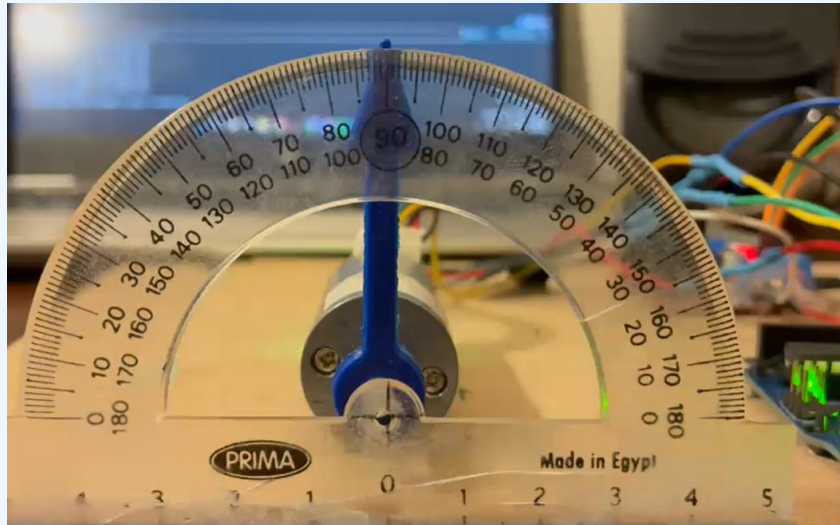


Figure 4: Pointer mounted on the motor output shaft and aligned with the protractor.

6 Wiring Connections

The hardware wiring connects the Arduino Uno, L298N motor driver, DC motor, encoder, and external 12 V power supply. The final control code uses the following pin definitions:

```
const int ENA_PIN    = 5;    // PWM speed control
const int IN1_PIN    = 7;    // Direction control input 1
const int IN2_PIN    = 6;    // Direction control input 2
const int ENCODER_A  = 2;    // Interrupt channel
const int ENCODER_B  = 3;    // Quadrature direction channel
```

6.1 Arduino to L298N Driver

Arduino Pin	L298N Pin	Function
D5	ENA	PWM speed control
D7	IN1	Direction control input 1
D6	IN2	Direction control input 2
GND	GND	Common ground

6.2 Encoder to Arduino

Encoder Signal	Arduino Pin	Function
Encoder VCC	5V	Encoder power supply
Encoder GND	GND	Encoder ground
Encoder Channel A (interrupt)	D2	Count update using interrupt on CHANGE
Encoder Channel B	D3	Direction detection

Encoder Signal	Arduino Pin	Function
(quadrature)		

6.3 Motor and Power Wiring

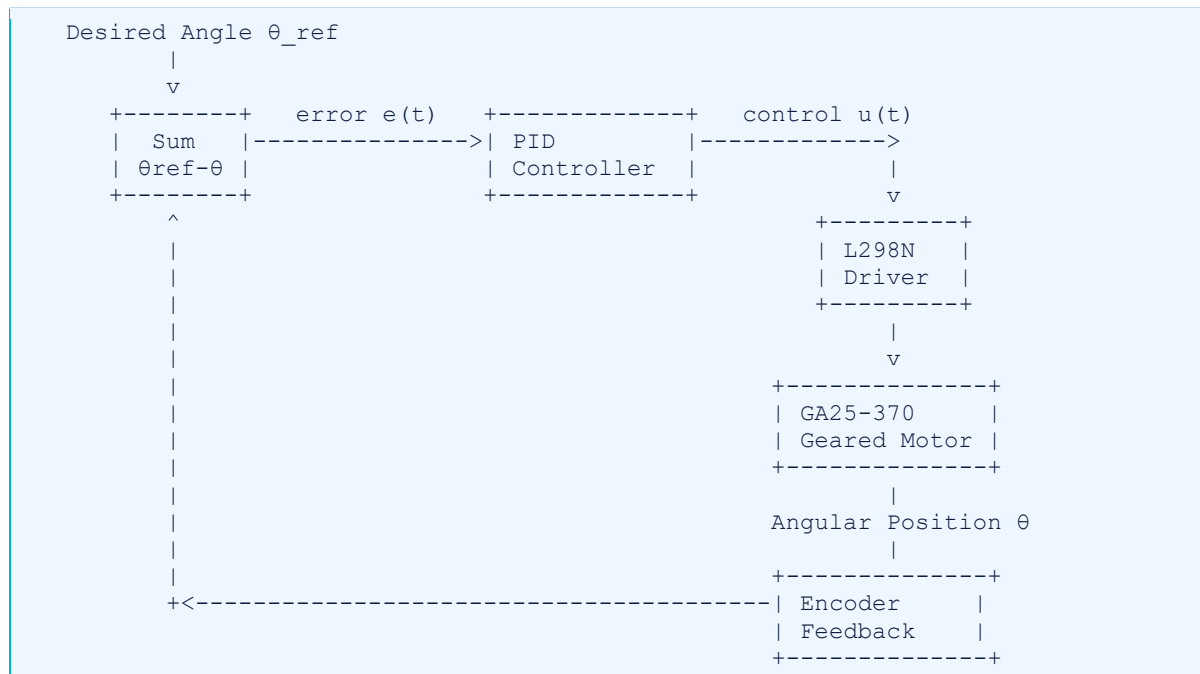
Connection	Description
Motor terminal 1	Connected to L298N OUT1 motor output
Motor terminal 2	Connected to L298N OUT2 motor output
12 V adapter positive	Connected to L298N 12 V input terminal
12 V adapter negative	Connected to L298N GND
Arduino GND	Connected to L298N GND and encoder GND (common ground)

A common ground is essential. The Arduino ground, L298N ground, encoder ground, and power supply ground must all be connected together. Without a common ground, Arduino control signals and encoder readings may become unstable or incorrect.

7 Closed-Loop Control System Description

7.1 Block Diagram

The project is implemented as a closed-loop position control system. The figure below shows the signal flow between all system elements:



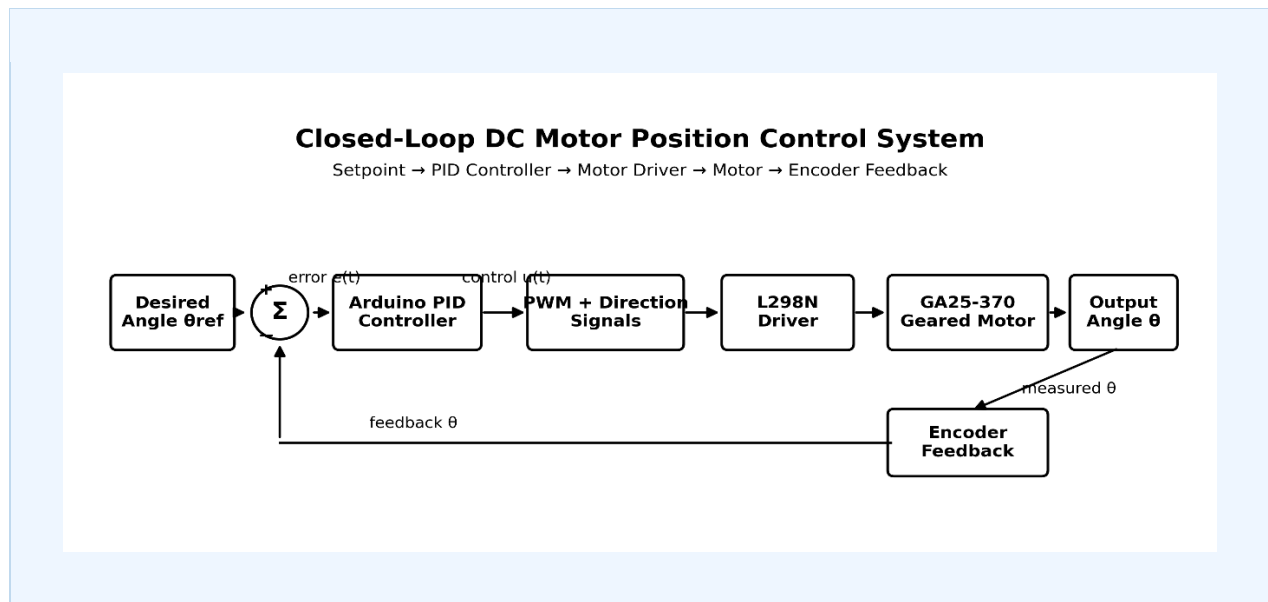


Figure 6: Closed-loop block diagram of the PID position control system.

7.2 Control System Elements

Control System Element	Implementation in This Project
Reference input	Desired relative angular displacement entered through Serial Monitor.
Controlled variable	Output shaft angular position in degrees.
Feedback sensor	Integrated incremental encoder.
Controller	PID controller implemented on Arduino Uno.
Manipulated variable	Motor drive command: direction + PWM duty cycle.
Final control element	L298N H-Bridge motor driver.
Actuator / plant	GA25-370 geared DC motor.
Visual verification	3D-printed pointer and protractor.

8 Encoder Feedback and PPR Calibration

The encoder is an incremental quadrature encoder integrated with the motor. It generates digital pulses as the motor shaft rotates. The Arduino counts these pulses using hardware interrupts. Since the encoder is incremental, it does not provide an absolute position after power-up, so the system uses a zeroing command at startup.

8.1 Zeroing Procedure

- Manually align the physical pointer with 0° on the protractor.
- Open the Arduino Serial Monitor.
- Type z and press Enter.
- The Arduino resets encoder_count to zero.
- Future angle readings are measured relative to this zero position.

The final PID implementation uses TOTAL_PPR = 995, and the angle is calculated as:

$$\theta = (\text{encoder_count} / 995) \times 360^\circ$$

8.2 PPR Estimation Procedure

The following subsection presents the original PPR estimation procedure used during motor characterization. In this stage, the encoder was counted using rising-edge interrupts, so the measured value represents the effective rising-edge PPR.

The encoder resolution is critical for accurate angular position and velocity feedback. For the GA25 motor with an integrated gearbox, the effective PPR at the output shaft is calculated as:

$$PPR_{\text{output}} = PPR_{\text{motor}} \times \text{GearRatio}$$

Given a base encoder resolution of 11 PPR and a gear ratio of ~46.8:1, the resulting resolution is ~515 pulses per revolution.

Analytical calculation of the nominal PPR value was performed to establish a reference baseline for validating experimental data.

After establishing the theoretical baseline, multiple experiments were held so that we can precisely measure how many pulses are produced while the motor's shaft spins a full revolution. Using the Arduino code that interrupts on the rising edge we calculated the PPR to be 497.5.

The discrepancy between the nominal (515) and measured (497.5) values is likely because manufacturers often round their gear ratios on spec sheets. A "46.8:1" gearbox is usually a complex train of gears with a true fractional ratio (e.g., if the true ratio is actually 45.2:1, then $11 \times 45.2 = 497.2$, which perfectly matches your data). For all subsequent controller tuning and state-space modeling, the empirically measured value of 497.5 PPR was utilized to ensure maximum accuracy.

Using the number of pulses that happen in a given sampling time e.g. 50ms we can accurately estimate the RPM of the motor which is essential for the parameter estimation phases.

For the final Arduino position-control implementation, the encoder interrupt mode was changed from RISING to CHANGE. This counts both rising and falling transitions of encoder Channel A, effectively doubling the count resolution. Therefore, the measured rising-edge value of 497.5 PPR corresponds to approximately 995 counts per output shaft revolution in the final code.

Accordingly, the final calibrated value used in the Arduino implementation was:

$$\text{TOTAL_PPR} = 995$$

This value was also verified experimentally using the physical pointer and protractor, and it provided the best agreement between the encoder-based angle calculation and the measured angular displacement.

9 Open-Loop Data Collection

Before closing the loop with PID control, open-loop data were collected to understand the motor response using the ParameterEstimation.ino sketch. The code applies a step or pulse input to the motor through the L298N driver and logs the motor speed in RPM. Encoder pulses are counted over a fixed time window, then converted to RPM.

The code is currently configured for pulse-response data collection with dynamic braking enabled. The RPM is calculated every 50 ms using:

$$\text{RPM} = (\text{current_pulses} / \text{TOTAL_PPR}) \times (60000 / \text{time_window_ms})$$

10 Motor Mathematical Model

The geared DC motor is modelled using standard armature-controlled DC motor equations. The electrical and mechanical equations are:

Electrical:	$V = R_a \cdot i + L_a \cdot (di/dt) + K_e \cdot \omega$
Mechanical:	$J \cdot (d\omega/dt) + b \cdot \omega = K_t \cdot i$
Position:	$\theta = \int \omega \, dt$

For the simplified model, armature inductance was neglected because the electrical dynamics are much faster than the mechanical dynamics. Assuming $K_t = K_e = K$, the model reduces to a cascade of gain blocks, a first-order mechanical transfer function, and an integrator.

Parameter	Value	Unit	Description
J	5.5744×10^{-6}	kg·m ²	Equivalent rotor/load inertia
K	0.020117	N·m/A (V·s/rad)	Motor constant ($K_e = K_t = K$)
R _a	6.2432	Ω	Armature resistance
b	2.2381×10^{-6}	N·m·s/rad	Viscous friction coefficient
L _a	Neglected	H	Armature inductance

11 Parameter Estimation

Parameter estimation was performed to obtain a usable mathematical model of the motor for simulation and PID tuning. The main steps were:

- Upload ParameterEstimation.ino to the Arduino.
- Apply a step or pulse PWM input to the motor.
- Count encoder pulses over fixed 50 ms intervals and convert to RPM.
- Export the serial data as CSV and import into MATLAB.
- Fit the motor parameters to match the measured rising response.
- Build the corresponding Simulink motor model.

Accurate estimation of the following parameters is essential for the transfer function model:

- Armature Resistance (R_a): Measured using a multimeter across the motor terminals while the rotor is locked.
- Armature Inductance (L_a): Determined through the electrical time constant response to a voltage step. (we assumed it to be zero)
- Back-EMF Constant (K_b): Calculated by measuring the generated voltage (V_{emf}) while back-driving the motor at a known angular velocity (ω): $K_b = \frac{V_{emf}}{\omega}$
- Torque Constant (K_t): In SI units, K_t is numerically equivalent to K_b (Nm/A).
- Combined Viscous Friction (b) and Rotor Inertia (J_{eq}): Derived from the mechanical time constant and steady-state speed analysis of the no-load step response.

Nominal Parameter Values

To establish a mathematically sound baseline for the parameter estimation algorithm, nominal

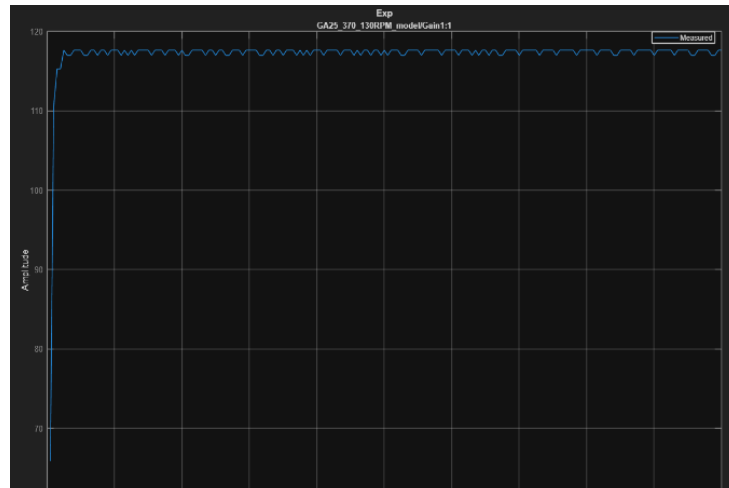
values were derived directly from the manufacturer's steady-state datasheet specifications (Operating Voltage = 12V, Stall Current = 1.8A, No-Load Speed = 130 RPM, No-Load Current = 0.07A, Gear Ratio = 46.8:1).

- **Armature Resistance (R_a): 6.67 , Omega**
 - Calculated via Ohm's Law at the stall condition, where angular velocity and Back-EMF are zero 12V / 1.8A).
- **Armature Inductance (L_a): 0**
 - Assumed to be zero, as the electrical time constant of a brushed DC motor of this scale is negligibly small compared to its mechanical time constant.
- **Motor Constant (K_b and K_t): 0.0181 V.s/rad**
 - Derived by subtracting the frictional voltage drop ($I_{nl} \cdot R_a$) from the supply voltage to find the true Back-EMF, and dividing by the internal angular velocity in rad/s. In SI units, K_b is numerically equivalent to K_t , so a unified constant K is used.
- **Viscous Friction (b): $2.27 \times 10^{-6} \text{ N.ms/rad}$**
 - Calculated by balancing the electromagnetic torque generated by the no-load current against the internal mechanical friction at maximum steady-state angular velocity $b = \frac{K_t \cdot I_{nl}}{\omega_{nl}}$
- **Rotor Inertia (J_{eq}): $8.8 \times 10^{-7} \text{ kg.m}^2$**
 - Calculated empirically by extracting the mechanical time constant (τ_m) from the physical open-loop step response data. This time constant was then algebraically unspooled using the known electrical resistance, torque constant, and viscous friction to isolate the true physical inertia of the system.

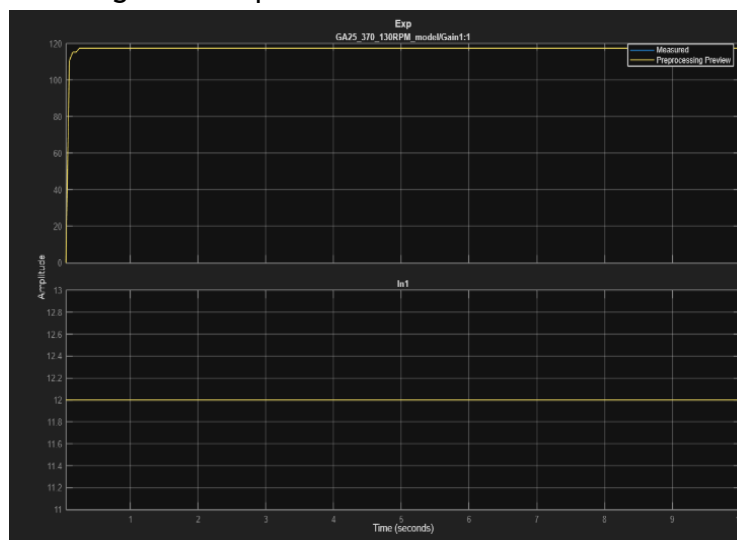
These analytically and empirically derived nominal values served as the highly constrained initial conditions for the computational parameter estimation algorithm. This ensured the solver converged on actual physical realities rather than mathematical artifacts.

Estimation Using Simulink

Using the measured Step Response shown in the given figure.



To get more accurate estimates of our parameters we replaced any noisy points with the final value average ~117.5 RPM to get the response below.



We used this data to estimate our parameters, and converging to the values in the figure below.

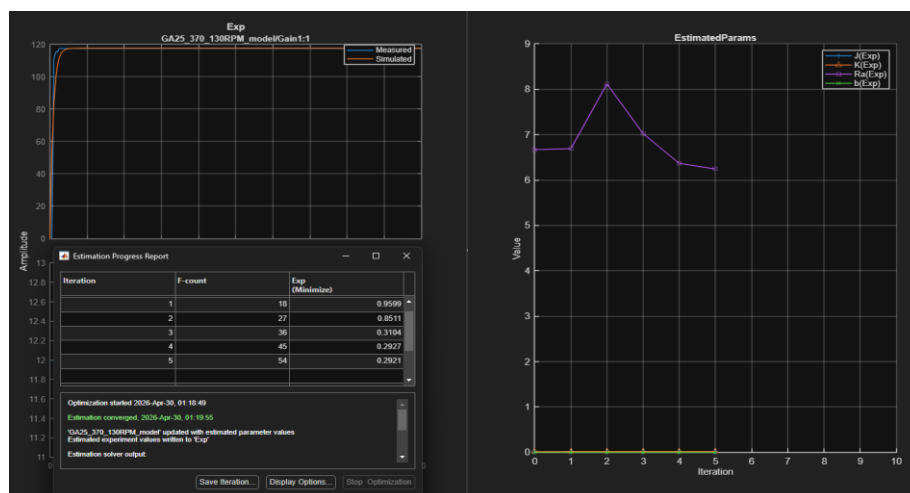


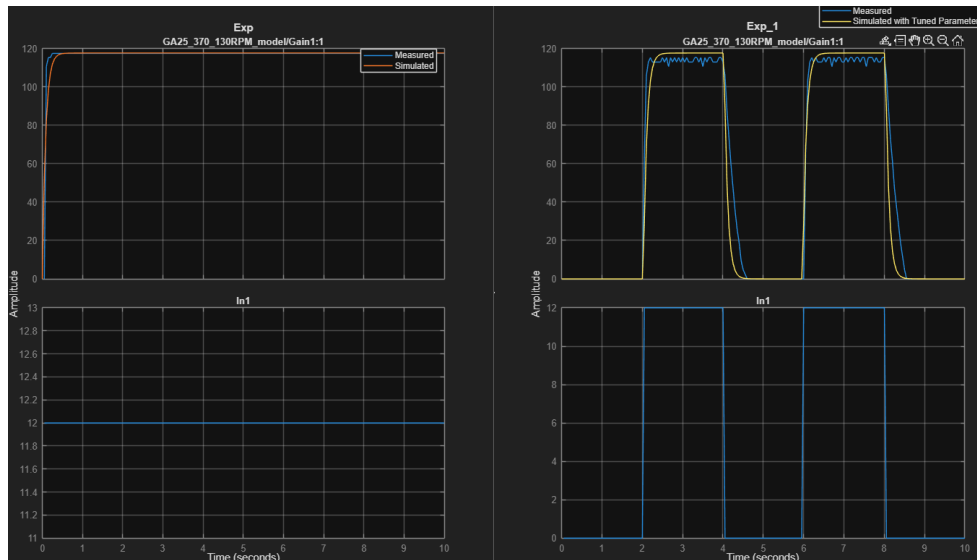
Figure 12: Comparison between measured and simulated motor response.

Model Validation and H-Bridge Dynamics

Following the parameter estimation phase, the second-order transfer function model was validated by comparing its simulated output against experimental pulse response data. The primary objective of this validation was to ensure the transient characteristics (rise time, settling time, and deceleration profile) aligned within acceptable tolerances.

Initial Discrepancy: High-Impedance Coasting

Initial validation attempts utilizing a square pulse input exhibited significant residuals, specifically during the deceleration phase of the pulse. The simulated model predicted a sharp, rapid deceleration, whereas the physical motor exhibited a prolonged, lazy coasting behavior.



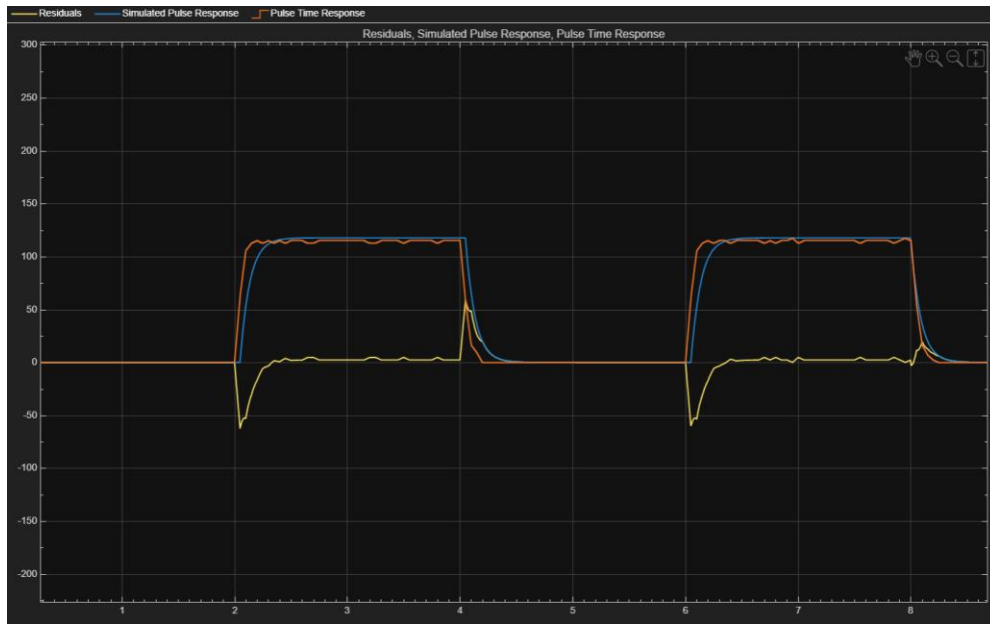
An analysis of the motor driver (H-bridge) circuitry revealed a fundamental mismatch between the physical hardware state and the assumptions governing the mathematical model. In the initial pulse test, the motor was stopped by driving the PWM (Enable) pin to zero. This action places the H-bridge into a high-impedance state, effectively open-circuiting the motor terminals. With the electrical circuit broken, current cannot flow, and the electromagnetic torque becomes zero. Consequently, the motor coasts to a stop governed entirely by its mechanical properties (inertia J_{eq} and friction b).

However, the continuous-time transfer function $G(s)$ intrinsically assumes a closed electrical circuit. When the simulated input voltage drops to 0V, the mathematical model assumes the spinning rotor's Back-EMF will push current through the armature resistance R_a generating a massive opposing torque that actively brakes the system.

Resolution: Implementing Dynamic Braking

To synchronize the physical hardware with the mathematical model, the microcontroller's logic was modified to implement dynamic braking. Rather than disabling the PWM pin to drop the voltage, the digital directional inputs (IN1 and IN2) were driven to identical logic levels (HIGH/HIGH or LOW/LOW) while the Enable pin remained active.

This configuration utilizes the H-bridge's internal MOSFETs to physically short the motor terminals together. This closes the armature circuit, allowing the Back-EMF to generate the braking current exactly as predicted by the transfer function.



Validation Results

Upon implementing dynamic braking, the physical pulse response tightly converged with the simulated output. The large residuals previously observed during deceleration were eliminated. This confirms that the parameters estimated from the step response (R_a , K_t , b , J_{eq}) are highly accurate and that the model is strictly valid provided the hardware is operated in a closed-circuit braking state.

Final Estimated Parameters

The computational parameter estimation algorithm, constrained by the nominal values and validated against the physical step response data, yielded the following final system parameters for the GA25 motor:

- **Rotor Inertia (J_{eq}):** $5.5744 \times 10^{-6} \text{ kg}\cdot\text{m}^2$
- **Motor Constant (K):** $0.020117 \text{ V}\cdot\text{s}/\text{rad}$
- **Armature Resistance (R_a):** 6.2432Ω
- **Viscous Friction (b):** $2.2381 \times 10^{-6} \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$
- **Armature Inductance (L_a):** 0 H

12 MATLAB/Simulink Simulation

MATLAB/Simulink was used to model the motor dynamics and study the effect of PID gains before hardware testing. The simulation allows safe testing of different controller parameters without risking unstable hardware behaviour.

12.1 Motor Model

The motor model represents the simplified DC motor equations using Simulink blocks. The model includes the applied voltage input, back-EMF feedback term ($K\omega$), armature resistance gain ($1/Ra$), torque constant gain (K), mechanical transfer function ($1/(Js+b)$), and an integrator to obtain angular position from angular speed.

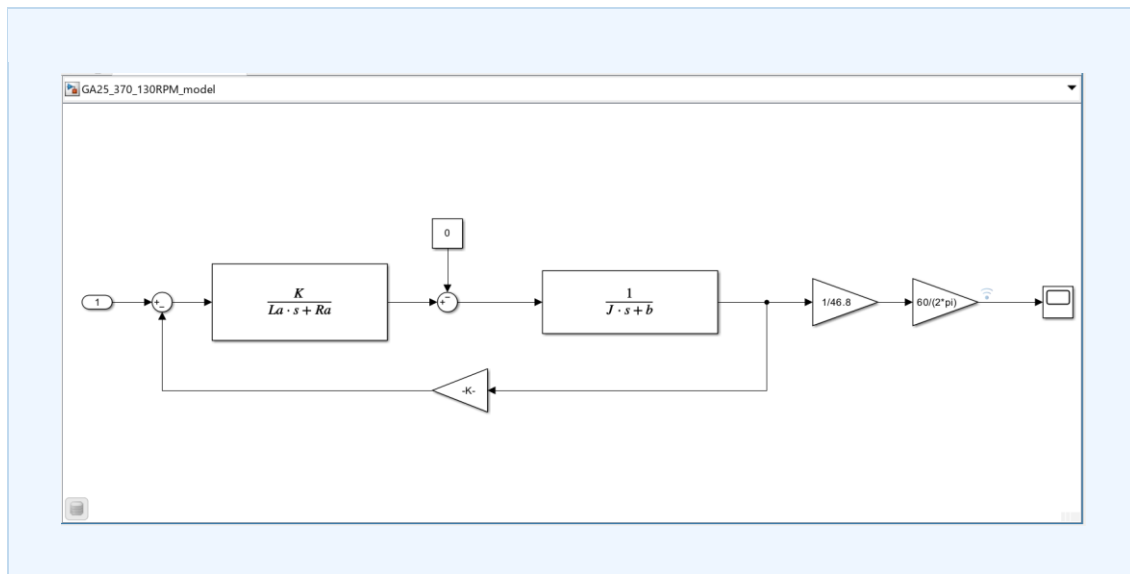


Figure 13: Simulink implementation of the simplified DC motor model.

12.2 PID Motor Model

A PID controller was added in an outer feedback loop. The angle setpoint is compared with the simulated motor angle, and the PID output is applied as the motor voltage command. The controller output is saturated between -12 V and $+12$ V to represent the physical supply voltage limitation.

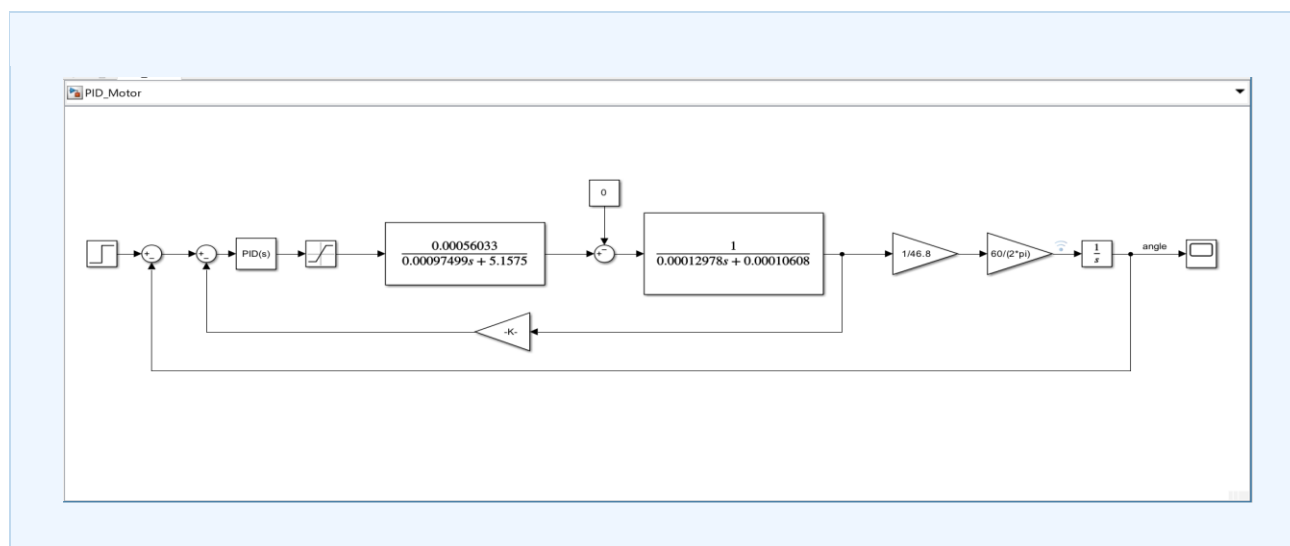


Figure 14: Simulink PID position control model.

12.3 Full System for Tuning

A full tuning system was built to compare the motor response with and without PID control and to help select suitable PID parameters. MATLAB PID tuning tools and manual adjustment were used to obtain starting controller gains, which were then refined on hardware.

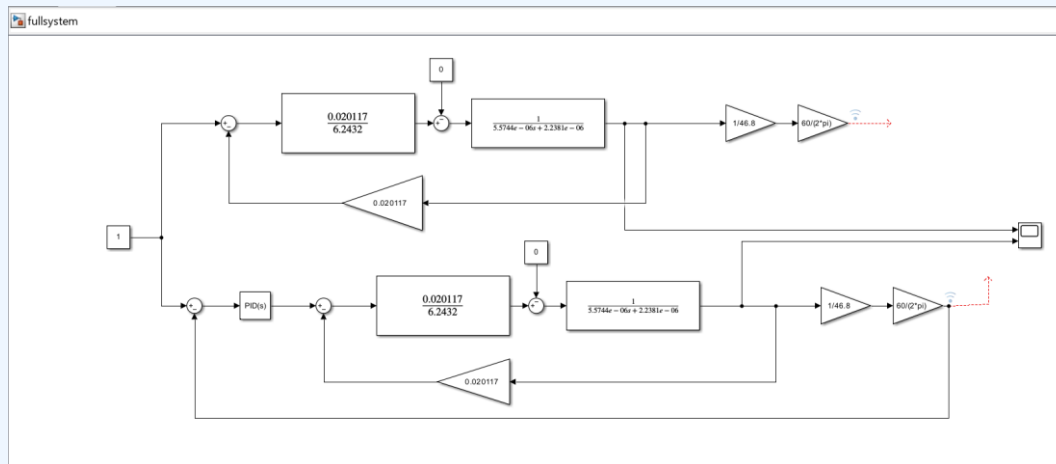


Figure 15: Full Simulink system used for PID tuning and response comparison.

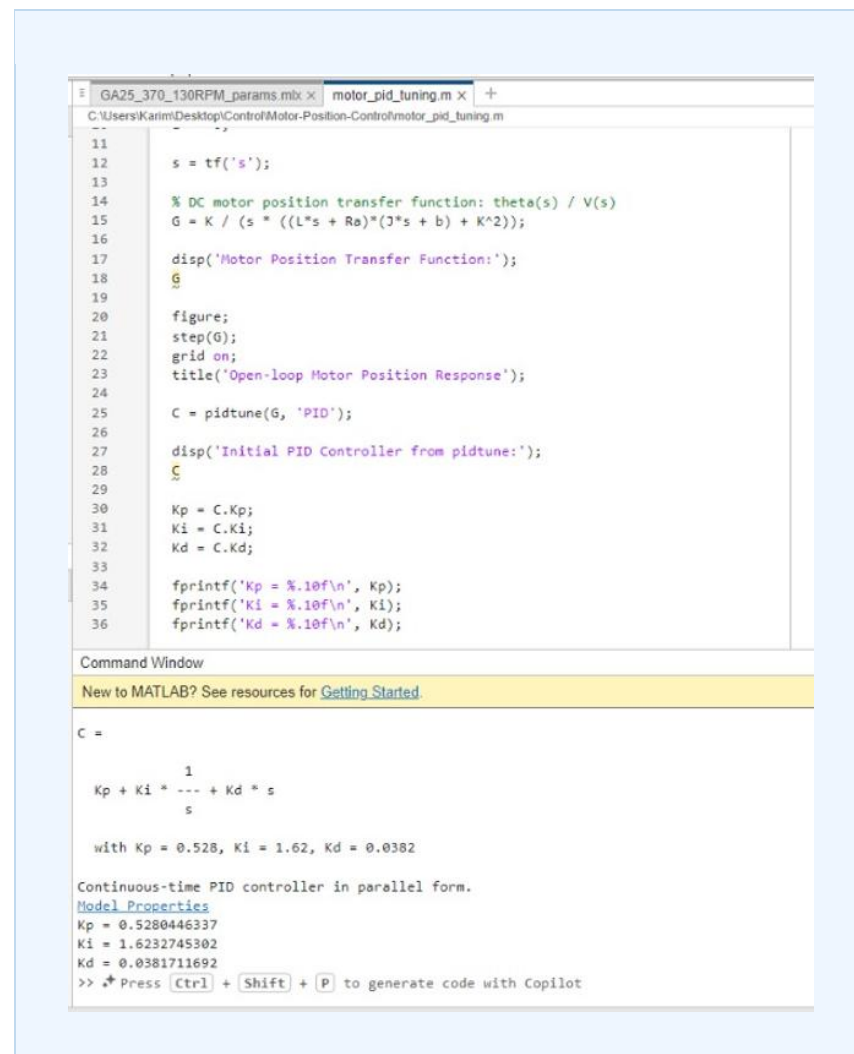


Figure 16: Simulated response after PID tuning.

12.4 Simulation vs. Hardware

The simulation provided a useful starting point, but hardware tuning was still required. The main causes of mismatch between simulation and real hardware were:

- Gearbox friction and stiction.

- Backlash in the geared motor.
- L298N voltage drop.
- Encoder quantisation.
- Sampling time of the digital controller.
- Motor dead zone at low PWM values.
- Mechanical pointer tolerance and alignment error.

13 PID Controller Design

The final controller is a discrete-time PID controller implemented on the Arduino Uno with a fixed sample time of $T_s = 20$ ms. At each control cycle:

```
e[k]   =  $\theta_{ref}[k] - \theta[k]$ 
I[k]   =  $I[k-1] + e[k] \times T_s$ 
D[k]   =  $(e[k] - e[k-1]) / T_s$ 
u[k]   =  $K_p \cdot e[k] + K_i \cdot I[k] + K_d \cdot D[k]$ 
u[k]   = clamp(u[k], -12, +12)
PWM    =  $(|u[k]| / 12) \times 255$ 
```

Gain	Value	Role in the Controller
Kp	0.035	Proportional — improves response speed.
Ki	0.4	Integral — eliminates steady-state error.
Kd	0.042	Derivative — improves damping, reduces overshoot.

13.1 Integral Anti-Windup

The final code limits the stored integral memory to prevent integral windup. This prevents the integral term from contributing more than approximately 3 V to the control output, improving stability and reducing overshoot after the motor starts moving.

```
float max_integral_memory = 3.0 / Ki;
integral = constrain(integral, -max_integral_memory, max_integral_memory);
```

13.2 Minimum PWM Compensation

The motor may not move at very low PWM values due to static friction and gearbox stiction. The final code raises any non-zero PWM below 45 up to the minimum threshold:

```
if (pwm > 0 && pwm < 45) {
    pwm = 45;
}
```

13.3 Target Tolerance and Settling Monitor

When the absolute error becomes smaller than $\pm 0.8^\circ$, the motor is stopped and the code enters settling mode. During settling mode the Arduino prints the target, angle, and error for 3 seconds, after which the system becomes ready for a new command.

14 Final Arduino Implementation

14.1 Main Features

- Encoder reading using hardware interrupt on CHANGE.
- Quadrature direction logic.
- Software zeroing using the z command.
- Relative angle commands using angle direction format.
- PID control loop with 20 ms sampling.
- Integral clamping to avoid windup.
- Output saturation at ± 12 V.
- PWM mapping from control voltage.
- Minimum PWM compensation (threshold = 45).
- Direction control through L298N IN1 and IN2 pins.
- Target tolerance detection ($\pm 0.8^\circ$).
- Settling monitoring for 3 seconds after reaching target.

14.2 Serial Command Interface

At startup, the Arduino prints instructions to the Serial Monitor at 115200 baud. The demonstration procedure is:

- Power the circuit.
- Open Serial Monitor at 115200 baud.
- Manually align the pointer to 0° on the protractor.
- Type z to set the current encoder count to zero.
- Send a relative command, for example 90 0.
- Observe the motor moving to the required angle.
- Wait until settling monitoring finishes, then send another command if needed.

14.3 Final Code Constants

Constant	Value	Purpose
TOTAL_PPR	995	Converts encoder counts to output shaft angle.
Kp	0.035	Proportional gain.
Ki	0.4	Integral gain.
Kd	0.042	Derivative gain.
sample_time_ms	20 ms	PID update period.
maxVoltage	12 V	Saturation limit.
targetTolerance	0.8°	Error band for target reached.
settlingDurationMs	3000 ms	Monitoring time after target reached.
Minimum PWM	45	Compensation for motor dead zone/stiction.

15 Experimental PID Tuning Trials

The PID controller was tuned experimentally after the simulation stage. Several trials were performed by changing the PID gains and observing the motor response. For each trial, the controlled variable (angular position of the motor output shaft) was plotted versus time.

15.1 Trial Summary Table

Trial	Setpoint	Kp	Ki	Kd	SS Error	Rise Time	Overshoot	Settling Time
Trial 1	90	0.10	0	0	Large	Fast	Very High	Long / Unstable
Trial 2	90	0.05	0	0.10	Moderate	Moderate	Low	Moderate
Trial 3	90	0.10	0.001	0.15	Small	Fast	Moderate	Medium
Trial 4	90	0.02	0.001	0.19	Very Small	Fast	Very Low	Short

15.2 Trial 1

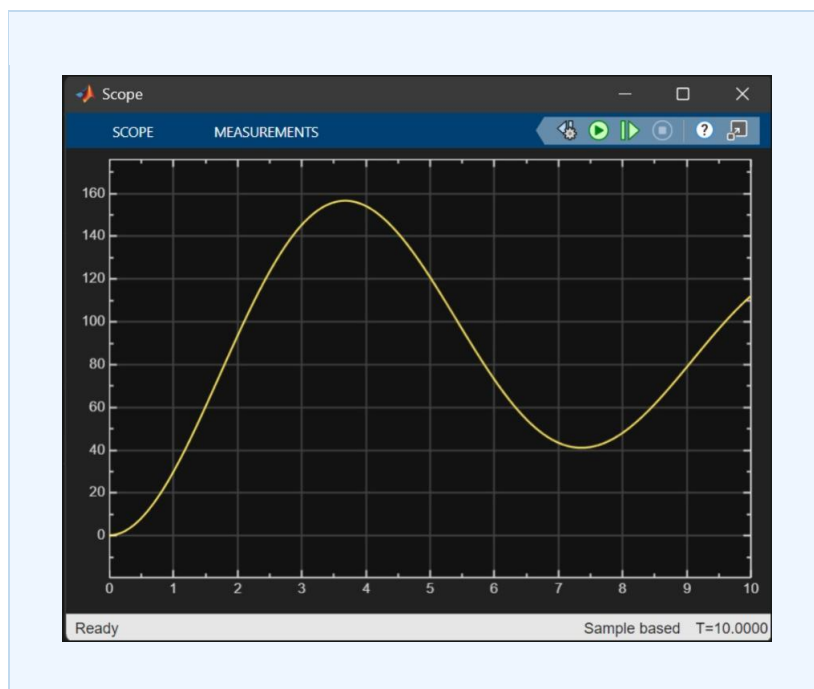


Figure 16: PID tuning trial 1: angular position versus time.

15.3 Trial 2

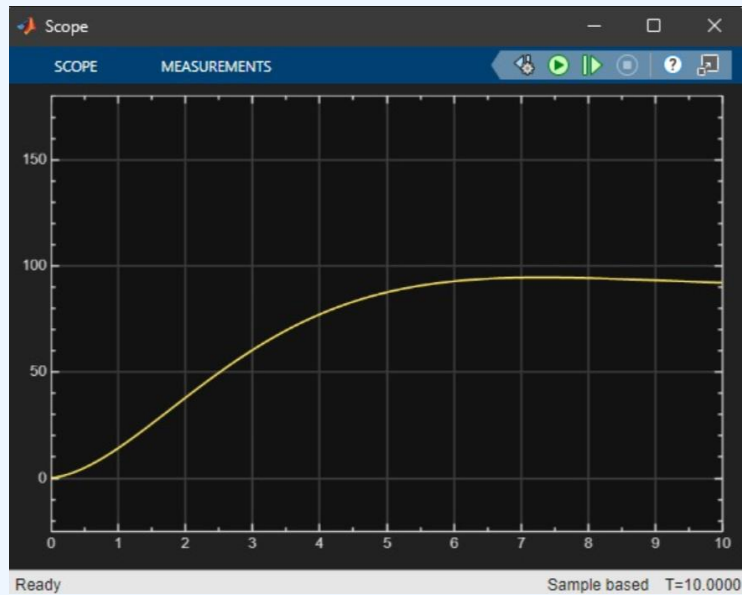


Figure 17: PID tuning trial 2: angular position versus time.

15.4 Trial 3

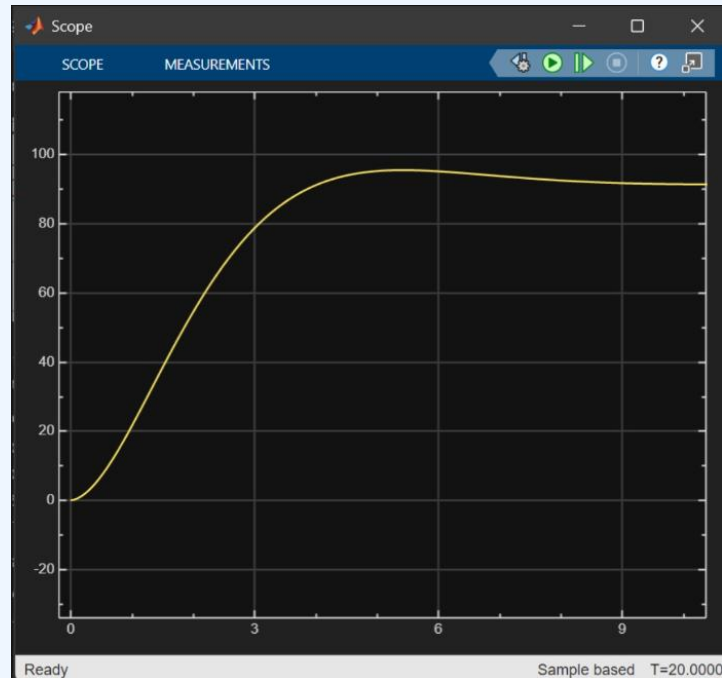


Figure 18: PID tuning trial 3: angular position versus time.

15.5 Trial 4

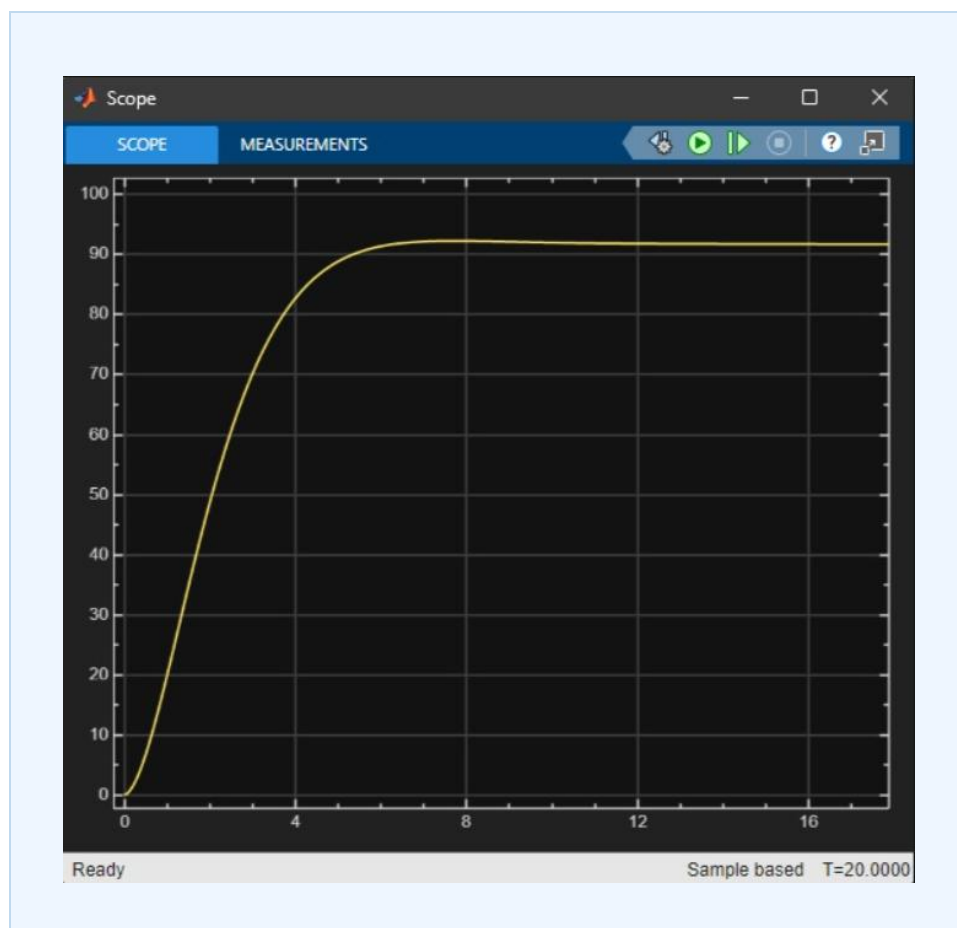


Figure 19: PID tuning trial 4: angular position versus time.

16 Final Response and Discussion

After comparing the PID tuning trials, the final selected parameters were $K_p = 0.035$, $K_i = 0.4$, and $K_d = 0.042$. These values were selected because they provided a practical balance between fast response, acceptable overshoot, reduced oscillation, and small steady-state error on the real hardware.

16.1 Final Response Metrics

Metric	Value
Final setpoint(s)	[Insert, e.g. 30°, 90°, 180°]
Final K_p	0.035
Final K_i	0.4
Final K_d	0.042

[FIGURE 21 — INSERT IMAGE HERE]

Final response graph of angle versus time after tuning. Use one graph with 30°, 90°, and 180° commands, or one clean final response at a selected setpoint.

Figure 21: Final tuned angular position response.

16.2 Discussion of Final Performance

The final response demonstrates that the encoder-based feedback system can accurately control the angular position of the geared DC motor. The proportional term improves response speed, the integral term reduces steady-state error, and the derivative term improves damping and reduces overshoot. The minimum PWM compensation helps the motor overcome static friction at low control outputs, while integral clamping prevents excessive accumulated error.

The remaining small steady-state error is mainly due to encoder resolution, gearbox backlash, friction, mechanical pointer alignment, and the tolerance band used in the code. The tolerance band is necessary because a real geared DC motor cannot practically hold an exact continuous angle without small oscillations or repeated corrections.

17 Problems Faced and Solutions

17.1 Encoder Wiring and Direction Issues

At the beginning, the encoder readings were not always consistent with the actual motor motion. This can happen if the encoder channels are swapped, if the motor direction is opposite to the assumed direction, or if the encoder count sign is reversed.

Solution: The encoder values were checked through the Serial Monitor while rotating the motor. The final code was written so that the encoder interrupt updates the count and the second quadrature channel determines the direction. The final direction logic was tested with the actual motor setup.

17.2 PPR Confusion

Different encoder PPR values can appear in datasheets or online references. Some values refer to the encoder motor shaft before the gearbox, while others refer to output shaft counts after the gearbox and quadrature decoding.

Solution: A practical PPR calibration test was performed using PPREstimation.ino. The final value was experimentally tuned to $TOTAL_PPR = 995$, which provided the best practical agreement between encoder-based angle calculation and the physical pointer/protractor measurement.

17.3 Model and Hardware Mismatch

The Simulink model represents the motor using simplified linear equations. The real motor includes nonlinear effects such as gearbox friction, stiction, backlash, voltage drops, and encoder quantisation.

Solution: Simulation was used only as a starting point for PID tuning. The final gains were tuned experimentally on hardware.

17.4 Falling Response Mismatch During Parameter Estimation

The measured falling response did not always match the same model used for the rising response, because when the input is removed, the driver may cause coasting or braking behaviour.

Solution: The rising response was emphasised during parameter estimation, because it better represents the relationship between applied input voltage/PWM and motor speed increase.

17.5 Motor Stiction and Dead Zone

The motor sometimes did not move when the controller output was too small, due to static friction and gearbox stiction at very low PWM values.

Solution: The final code uses minimum PWM compensation. If the calculated PWM is between 1 and 44, it is raised to 45, ensuring the motor has enough torque to overcome the dead zone.

17.6 Integral Windup

When the motor is stuck due to friction, the error can remain large and the integral term can accumulate excessively, causing overshoot once the motor starts moving.

Solution: The final code clamps the integral memory so that the integral term cannot contribute more than approximately 3 V to the control output. This prevents the integral term from dominating the controller.

17.7 Incremental Encoder Zero Position

The encoder is incremental, so it does not know the absolute angle when the system starts. If the Arduino is reset, the encoder count returns to zero even if the pointer is not physically at zero.

Solution: The final code includes a zeroing command. The user manually aligns the pointer to zero degrees and types z to set the current position as the software zero.

18 Conclusion

A closed-loop angular position control system for a geared DC motor was successfully designed and implemented. The final system uses an Arduino Uno, L298N H-Bridge driver, GA25-370 geared DC motor with integrated encoder, and a 3D-printed pointer for visual verification.

The encoder feedback was used to measure the motor position by counting pulses and converting them to degrees using an experimentally calibrated value of $TOTAL_PPR = 995$. A PID controller was implemented on the Arduino with a 20 ms sample time and final gains $K_p = 0.035$, $K_i = 0.4$, and $K_d = 0.042$.

Open-loop data collection and parameter estimation were performed to build a motor model. MATLAB/Simulink simulations were used to understand the motor dynamics and support PID tuning. The final controller was refined experimentally to handle real hardware effects such as stiction, friction, backlash, encoder quantisation, and driver voltage limitations.

The final implementation supports relative angle commands, zero calibration, integral anti-windup, minimum PWM compensation, and settling monitoring. The physical pointer and protractor were used to verify the achieved angular position. Overall, the project demonstrates the practical implementation of a closed-loop PID position control system and shows how simulation, modelling, and experimental tuning are combined in real control-system design.

Appendix A Final Arduino PID Code (PID.ino)

```
// Best Yet - Relative Angle + Direction + Zero Command
// With settling monitor after reaching target

// --- MOTOR & L298N PINS ---
const int ENA_PIN = 5;
const int IN1_PIN = 7;
const int IN2_PIN = 6;

// --- ENCODER PINS ---
const int ENCODER_A = 2;
const int ENCODER_B = 3;

// --- ENCODER SPECIFICATION ---
const float TOTAL_PPR = 995;

// --- PID GAINS ---
float Kp = 0.035;
float Ki = 0.4;
float Kd = 0.042;

// --- CONTROL SETTINGS ---
float setpoint = 0;
const float maxVoltage = 12.0;
const int sample_time_ms = 20;
const float targetTolerance = 0.8;

// --- STATE ---
bool commandReceived = false;
bool settlingMode = false;
unsigned long settlingStartTime = 0;
const unsigned long settlingDurationMs = 3000;

// --- VARIABLES ---
volatile long encoder_count = 0;
float error = 0;
float previous_error = 0;
float integral = 0;
unsigned long previous_time = 0;

void setup() {
    Serial.begin(115200);
    pinMode(ENA_PIN, OUTPUT);
    pinMode(IN1_PIN, OUTPUT);
    pinMode(IN2_PIN, OUTPUT);
    pinMode(ENCODER_A, INPUT_PULLUP);
    pinMode(ENCODER_B, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(ENCODER_A), updateEncoder, CHANGE);
    stopMotor();
    previous_time = millis();
    Serial.println("Ready.");
    Serial.println("First move pointer to real 0 degrees, then type: z");
    Serial.println("Then enter: angle direction");
    Serial.println("Example: 90 0 -> move +90 from current position");
    Serial.println("Example: 90 1 -> move -90 from current position");
}

void loop() {
    if (!commandReceived && Serial.available() > 0) {
        char firstChar = Serial.peek();
        if (firstChar == 'z' || firstChar == 'Z') {
            Serial.read();
            noInterrupts();
            encoder_count = 0;
            interrupts();
            setpoint = 0;
            integral = 0;
            previous_error = 0;
            settlingMode = false;
            Serial.println("Current position is now ZERO.");
            clearSerialBuffer();
            return;
        }
    }
    float inputAngle = Serial.parseFloat();
    int direction = Serial.parseInt();
}
```

```

    if (inputAngle > 0 && (direction == 0 || direction == 1)) {
        float currentAngle = getCurrentAngle();
        if (direction == 0) { setpoint = currentAngle + inputAngle; }
        else { setpoint = currentAngle - inputAngle; }
        commandReceived = true;
        settlingMode = false;
        integral = 0;
        previous_error = 0;
        previous_time = millis();
        clearSerialBuffer();
    }
    return;
}

if (!commandReceived) { stopMotor(); return; }

unsigned long current_time = millis();
if (current_time - previous_time >= sample_time_ms) {
    float dt = (current_time - previous_time) / 1000.0;
    previous_time = current_time;
    float angle = getCurrentAngle();
    error = setpoint - angle;

    if (abs(error) < targetTolerance && !settlingMode) {
        stopMotor();
        settlingMode = true;
        settlingStartTime = millis();
        Serial.println("Target reached. Monitoring settling...");
    }
    if (settlingMode) {
        stopMotor();
        if (millis() - settlingStartTime >= settlingDurationMs) {
            settlingMode = false;
            commandReceived = false;
            Serial.println("Movement finished.");
        }
        return;
    }

    float derivative = (error - previous_error) / dt;
    integral += error * dt;
    float max_integral_memory = 3.0 / Ki;
    integral = constrain(integral, -max_integral_memory, max_integral_memory);
    float controlVoltage = (Kp * error) + (Ki * integral) + (Kd * derivative);
    controlVoltage = constrain(controlVoltage, -maxVoltage, maxVoltage);
    int pwm = (int)((abs(controlVoltage) / maxVoltage) * 255.0);
    pwm = constrain(pwm, 0, 255);
    if (pwm > 0 && pwm < 45) { pwm = 45; }
    driveMotor(controlVoltage, pwm);
    previous_error = error;
}

float getCurrentAngle() {
    noInterrupts();
    long count_copy = encoder_count;
    interrupts();
    return (count_copy / TOTAL_PPR) * 360.0;
}

void updateEncoder() {
    int stateA = digitalRead(ENCODER_A);
    int stateB = digitalRead(ENCODER_B);
    if (stateA == stateB) { encoder_count--; }
    else { encoder_count++; }
}

void driveMotor(float controlVoltage, int pwm) {
    if (controlVoltage > 0) { digitalWrite(IN1_PIN, LOW); digitalWrite(IN2_PIN, HIGH); }
    else { digitalWrite(IN1_PIN, HIGH); digitalWrite(IN2_PIN, LOW); }
    analogWrite(ENA_PIN, pwm);
}

void stopMotor() {
    digitalWrite(IN1_PIN, LOW);
    digitalWrite(IN2_PIN, LOW);
    analogWrite(ENA_PIN, 255);
}

void clearSerialBuffer() {
    while (Serial.available() > 0) { Serial.read(); }
}

```

Appendix B PPR Estimation Code (PPREstimation.ino)

```
const int PWM_PIN = 5;
const int IN1_PIN = 7;
const int IN2_PIN = 6;
const int ENCODER_A = 2;
const int ENCODER_B = 3;

volatile long pulseCount = 0;
bool isRunning = true;

void setup() {
  Serial.begin(9600);
  pinMode(PWM_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);
  pinMode(ENCODER_A, INPUT_PULLUP);
  pinMode(ENCODER_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCODER_A), countPulse, RISING);
  digitalWrite(IN1_PIN, HIGH);
  digitalWrite(IN2_PIN, LOW);
  analogWrite(PWM_PIN, 50);
  Serial.println("Motor is running...");
  Serial.println("Type ANY letter and hit ENTER when it does 1 full spin.");
}

void loop() {
  if (Serial.available() > 0 && isRunning == true) {
    analogWrite(PWM_PIN, 0);
    digitalWrite(IN1_PIN, LOW);
    digitalWrite(IN2_PIN, LOW);
    Serial.println("--- MOTOR STOPPED ---");
    Serial.print("Pulses for 1 Revolution (CPR): ");
    Serial.println(pulseCount);
    while (Serial.available() > 0) { Serial.read(); }
    isRunning = false;
  }
}

void countPulse() { pulseCount++; }
```


Appendix C Parameter Estimation Code (ParameterEstimation.ino)

```

#define PULSE          1
#define STEP           0
#define DYNAMIC_BRAKING 1

const int ENA_PIN      = 5;
const int IN1_PIN      = 7;
const int IN2_PIN      = 6;
const int ENCODER_A    = 2;
const int ENCODER_B    = 3;

const float TOTAL_PPR  = 510;
const int   time_window = 50;
unsigned long pulse_window = 2000;

volatile long pulse_count = 0;
unsigned long previous_time = 0;
unsigned long pulse_previous_time = 0;
bool isLow = 1;

void setup() {
  Serial.begin(115200);
  pinMode(ENA_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);
  pinMode(ENCODER_A, INPUT_PULLUP);
  pinMode(ENCODER_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCODER_A), updateEncoder, RISING);
  Serial.println("Time(ms),Volt,RPM");
  digitalWrite(IN1_PIN, HIGH);
  digitalWrite(IN2_PIN, LOW);
  if (STEP) { analogWrite(ENA_PIN, 255); }
  else if (PULSE) { analogWrite(ENA_PIN, 0); }
}

void loop() {
  unsigned long current_time = millis();
  unsigned long pulse_time   = millis();
  if (current_time - previous_time >= time_window) {
    noInterrupts();
    long current_pulses = pulse_count;
    pulse_count = 0;
    interrupts();
    float revolutions = (float)current_pulses / TOTAL_PPR;
    float rpm = revolutions * (60000.0 / time_window);
    Serial.print(current_time);
    Serial.print(',');
    Serial.print(isLow ? 0 : 5);
    Serial.print(',');
    Serial.println(rpm);
    previous_time = current_time;
  }
  if (pulse_time - pulse_previous_time > pulse_window && PULSE) {
    if (isLow) {
      analogWrite(ENA_PIN, 255);
      if (DYNAMIC_BRAKING) { digitalWrite(IN1_PIN, HIGH); digitalWrite(IN2_PIN, LOW); }
      isLow = !isLow;
    } else {
      if (DYNAMIC_BRAKING) { analogWrite(ENA_PIN, 255); digitalWrite(IN1_PIN, LOW); }
      digitalWrite(IN2_PIN, LOW);
    }
    else { analogWrite(ENA_PIN, 0); }
    isLow = !isLow;
  }
  pulse_previous_time = pulse_time;
}

void updateEncoder() { pulse_count++; }

```